

PRIORITY QUEUE (heap)

```
priority_queue<int> pq; // max-heap default
pq.push(x); pq.top(); pq.pop(); // top()=largest
priority_queue<int, vector<int>, greater<int>> minpq; // min-heap version
priority_queue<pair<int,int>> pq2; // max-heap of pairs, compares .first then .second
pq2.push({5,1}); pq2.push({3,2}); pq2.push({5,0}); pq2.top(); //{5,1} – highest first
// min-heap of pairs
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> minpq2;
```

Demo: find the k largest elements in an array

```
priority_queue<int, vector<int>, greater<int>> minHeap;
for (int x : a) { minHeap.push(x); if (minHeap.size() > k) minHeap.pop(); }
// minHeap now holds the k largest elements of a[]
```

Demo: merge top elements from multiple sorted streams

```
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
// {value, streamIndex}
for (int i=0;i<k;i++) pq.push({streams[i][0], i});
while (!pq.empty()) { auto [val,idx]=pq.top(); pq.pop();
/* push next from stream idx */ }
```

RECURSION

```
// every recursive function needs a base case – without one it recurses
// forever and crashes (stack overflow)
int solve(int n) {
    if (n == 0) return baseCaseValue; // base case – always needed
    return combine(n, solve(n - 1)); // recursive call on smaller input
}
```

Demo: factorial

```
long long fact(int n) { if (n<=1) return 1; return n*fact(n-1); }
```

Demo: Fibonacci with memoization (avoids exponential blowup)

```
map<int,long long> memo;
long long fib(int n) {
    if (n<=1) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}
```

Demo: recursive sum of digits

```
int digitSum(int n) { if (n==0) return 0; return n%10 + digitSum(n/10); }
```

Demo: recursive binary search (recursion applied to searching)

```
int bsearch(vector<int>& a, int l, int r, int target) {
    if (l > r) return -1;
    int mid = l + (r-l)/2;
    if (a[mid] == target) return mid;
    else if (a[mid] < target) return bsearch(a, mid+1, r, target);
    else return bsearch(a, l, mid-1, target);
}
```